

# Evaluating SIMD-Vectorized Query Algorithms on Skewed and Non-Uniform Workloads

Brian Heckel  
UCID: 30165243

Ryan Khoshkbarchi  
UCID: 30206221

## 1 Abstract

This project aims to design, implement, and evaluate a custom query execution engine that leverages explicit SIMD SSE intrinsics to improve both the performance and energy efficiency of common OLAP operations. Specifically, we focus on column-oriented storage layouts (using Apache Parquet [4] and Arrow [2]) to maximize memory locality. By leveraging SIMD and its ability to perform operations on multiple data elements in parallel, we aim to surpass the performance of scalar execution for core operators such as Scans, Filters, and Aggregations.

## 2 Introduction

In the era of big data, Online Analytical Processing workloads have become increasingly read-intensive, which requires database systems to scan, filter, and aggregate billions of rows efficiently. Modern CPUs have evolved to include Single Instruction, Multiple Data capabilities such as AVX2 and AVX-512; however, traditional row-oriented database execution models often do not take advantage of these hardware features. A key novelty of this project is the integration of energy efficiency metrics. As data centers consume an increasing percentage of global power, performance per watt has become an important metric that many hardware and software vendors are beginning to highlight in their reports. In this project, we investigate the Race-to-sleep hypothesis, which theorizes that the high instantaneous power draw of AVX instructions is offset by the reduction in execution time, resulting in net energy savings.

## 3 Literature Review

Our research is based on three distinct pillars of database system design: the foundational execution models, the shift to vectorized processing in modern analytical engines, and instruction optimizations to mitigate control flow divergence.

### 3.1 Foundational Execution Models

The standard architecture for database query execution is the Volcano Iterator Model, formally described by Graefe [9, 10]. In this model, query

operators are arranged in a tree structure, passing a single tuple (row) at a time from child to parent via a virtual `Next()` function.

### 3.2 Vectorization & Columnar Storage

As explored by Sompolski et al. [26], vectorization processes data in batches rather than single rows. This architecture can be seen in modern embedded analytical databases like DuckDB [24], which utilizes a vectorized engine to increase efficiency on personal computing hardware. Furthermore, the adoption of column-oriented memory formats, such as Apache Arrow [2], enhances this approach by arranging data in contiguous blocks of memory which can minimize cache misses during scans and aggregations.

### 3.3 Algorithmic Optimizations

Beyond basic scans, specific research has focused on optimizing some operators. Kim et al. [16] revisited the trade-offs between Sort-Merge and Hash Joins on modern processors, finding that SIMD-width improvements often favor hash-based approaches due to easier parallelization of probe phases. Similarly, Lemire et al. [18] and Inoue et al. [11] outlined that the set intersection could be accelerated by orders of magnitude using specialized SIMD compression and comparison instructions. This gave us some inspiration for our design of the `Filter` operator as we will present later.

## 4 Comparison with Background and Critical Analysis

Although the classical Volcano model [9] remains the standard for row-oriented databases due to its simplicity and flexibility of implementation, we criticize it for being inefficient on modern hardware. Our project adopts the explicit vectorization strategy seen in state-of-the-art engines like DuckDB [24] in order to achieve better performance, but we extend this work by specifically evaluating the SIMD optimized algorithms on skewed and non-uniform datasets while also testing the Race-to-Sleep hypothesis. This hypothesis states that while AVX instructions draw higher instantaneous power, the reduction in execution time, achieved through SIMD, allows the hardware to return to a low-power idle state sooner, which results in net energy savings. This is a critical metric often overlooked in pure performance benchmarks.

## 5 Our Methodology

Our methodology is based on the traditional Volcano-style [9] iterator model where each operator exposes a `Next()` function, adapted for vectorized execution. Rather than processing a single tuple at a time, our engine processes batches of tuples, referred to as `DataChunks`.

### 5.1 Engine Architecture

The engine is built in C++20 and uses the Apache Arrow library [2]. The architecture follows a pull-based pipeline as shown in the following figure.

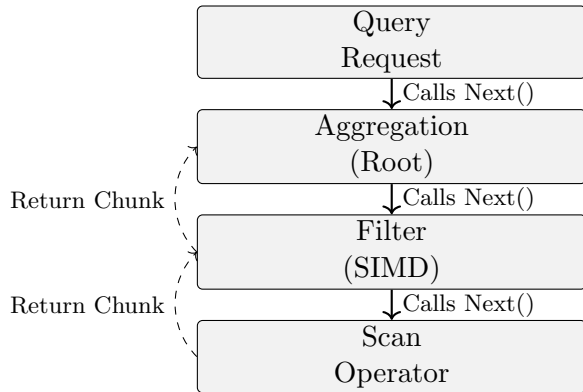


Figure 1: Volcano Model Control Flow

Data is initially stored in Apache Parquet files

in secondary memory (disk). Our engine implements two versions of the `SCAN` operator. The original `ScanOperator` loads and decodes compressed data blocks from secondary memory into Arrow record chunks (`DataChunks`). In order to strictly measure algorithmic efficiency, we also implement a `MemoryScanOperator` which fetches pre-loaded data directly from RAM. This bypasses inefficient disk I/O during benchmarking. In the next step of the pipeline, a tree of operators processes these chunks. Each operator implements a `Next()` interface that returns a `DataChunk`. In our operator implementations, we utilize Intel SSE/AVX intrinsics [13] (`immintrin.h`) to take advantage of SIMD.

### 5.2 Algorithmic Approach

Depending on the type of operator, we use two different SIMD strategies. For operations like filtering or projection, we load data into AVX registers and apply element-wise operations. This is especially useful for aggregate functions like `SUM` and `MIN` where we can take advantage of vertical vectorization and iteratively find the partial sum or min of 16 integers in a single instruction.

For aggregations, we maintain running partial sums in vector registers. When a chunk is finished, we collapse the vector into a single scalar value using a log-step reduction that involves shuffles and additions.

We created a customized `BenchmarkRunner` to compute the effective memory bandwidth of each algorithm (GB/s) as a stand-in for energy efficiency.

## 6 Effective Memory Bandwidth (Throughput)

Our primary metric is **Effective Memory Bandwidth** (GB/s) which measures the rate at which data is consumed from RAM by the computation unit.

$$T\text{-put (GB/s)} = \frac{\text{Total Input Bytes}}{\text{Execution Time (s)}}$$

The throughput metric shows how close the software is to saturating the physical capacity of the RAM channel. A high throughput enables the

vector units to function at their full potential by keeping the CPU’s arithmetic logic units (ALUs) occupied with a constant flow of data.

## 7 Energy Efficiency and the Race to Sleep

Based on the fundamentals of power consumption in contemporary Intel CPUs, we use the throughput metric as a reliable stand-in for energy efficiency as it is much simpler to calculate and does not require any proprietary libraries like Intel PCM [15].

1. **Static Power ( $P_{\text{static}}$ ):** The constant power draw required to keep the system’s cache, memory controller, and clock tree operational, even when the CPU core is idle. This power is paid by the system for the entire duration of the query.
2. **Dynamic Power ( $P_{\text{dynamic}}$ ):** The extra power consumed when execution units (like the AVX ALUs) switch on to perform work.

The **Race to Sleep** principle dictates that an algorithm is more energy-efficient if it completes its work faster, even if it draws more *instantaneous* power (higher  $P_{\text{dynamic}}$ ). Scalar execution typically results in low utilization, lower  $P_{\text{dynamic}}$ , but high Time. The system pays the high  $P_{\text{static}}$  cost for a long duration. On the other hand, vectorized execution results in high utilization, higher  $P_{\text{dynamic}}$ , but drastically reduced Time. The total Energy consumed is minimized because the system spends less time in the active, high-power state.

## 8 Implementation, Experimentation, and Challenges

Over the course of the four-month project period, we have successfully implemented and designed the core engine framework and five major operators: FILTER, SUM, MIN, the HashJoin, and the CuckooHashJoin.

### 8.1 The Filter Operator

The `FilterOperator` implements a relational algebra selection scan ( $\sigma_{\text{predicate}}$ ). The simple

scalar implementation iterates row-by-row with a standard `if` branch as shown in Algorithm 1.

---

#### Algorithm 1 Scalar Filter Operation

---

```

1: procedure FILTERSCALAR(Chunk, Thresh)
2:   InputColumn  $\leftarrow$  Chunk.GetColumn(0)
3:   ResultVector  $\leftarrow$  EmptyArray
4:   RowCount  $\leftarrow$  Length(InputColumn)
5:   for i  $\leftarrow$  0 to RowCount - 1 do
6:     CurrentValue  $\leftarrow$  InputColumn[i]
7:     if CurrentValue > Thresh then
8:       ResultVector.Append(CurrentValue)
9:     end if
10:  end for
11:  return MakeChunk(ResultVector)
12: end procedure

```

---

The AVX implementation is quite complex in comparison, requiring explicit intrinsic calls to minimize branching overhead, as seen in Algorithm 2.

---

#### Algorithm 2 AVX2 Vectorized Filter

---

```

1: procedure APPLYAVX2FILTER(Chunk, Threshold)
2:   Input  $\leftarrow$  Chunk.GetColumn(0).Raw()
3:   Count  $\leftarrow$  Chunk.Length()
4:   Output  $\leftarrow$  AllocateArray(Count)
5:   OutIdx  $\leftarrow$  0
6:   // Broadcast threshold to vector reg
7:   VecThresh  $\leftarrow$  _mm256_set1(Thresh)
8:   i  $\leftarrow$  0
9:   while i  $\leq$  Count - 8 do
10:    VecData  $\leftarrow$  _mm256_load(Input + i)
11:    VecMask  $\leftarrow$ 
      _mm256_cmpgt(VecData, VecThresh)
12:    Mask  $\leftarrow$  _movemask(VecMask)
13:    if Mask = 0 then
14:      // Skip
15:    else if Mask = 0xFF then
16:      MemCpy(Output + OutIdx, Input + i, 8)
17:      OutIdx  $\leftarrow$  OutIdx + 8
18:    else
19:      _storeu(TempBuf, VecData)
20:      for j  $\leftarrow$  0 to 7 do
21:        if (Mask  $\gg$  j) & 1 then
22:          Output[OutIdx]  $\leftarrow$ 
            TempBuf[j]
23:          OutIdx  $\leftarrow$  OutIdx + 1
24:        end if
25:      end for
26:    end if
27:    i  $\leftarrow$  i + 8
28:  end while
29:  return MakeBatch(Output, OutIdx)
30: end procedure

```

---

### 8.2 The Aggregation Operators

We chose to start by implementing the aggregate SUM and MIN operators due to their high vectorizability. In the first stage, the algorithm loops over the input array, iteratively updating a partial running sum in an AVX256 register by adding 8 integers at a time using

`_mm256_add_epi32`. This significantly reduces loop overhead as we can loop in batches of 8 integers at a time. A helper function, `HSum256` is then used to reduce the final register and store the final sum in the first lane of the AVX128 register.

### 8.3 Hash Join Operators

Join operations are fundamental to relational query processing, and hash joins are particularly well-suited for OLAP workloads due to their  $O(n + m)$  average-case complexity [7]. We implemented two hash join variants: Linear Probing and Cuckoo Hashing, each offering different trade-offs for SIMD optimization.

#### 8.3.1 Linear Probing Hash Join

Our Linear Probing implementation follows the classic build-probe paradigm [6]. During the build phase, we construct a hash table from the smaller (build) relation using a multiply-shift hash function based on the golden ratio prime (2654435769). We maintain a 50% load factor to minimize probe chain lengths.

The AVX2 implementation accelerates the probe phase using `_mm256_i32gather_epi32` to fetch 8 hash table entries in parallel. Algorithm 3 shows the vectorized probe logic.

---

#### Algorithm 3 AVX2 Linear Probing Hash Join (Probe)

---

```

1: procedure AVX2PROBE(ProbeKeys, HT)
2:   vPrime  $\leftarrow$  _mm256_set1(2654435769)
3:   vMask  $\leftarrow$  _mm256_set1(HT.Mask)
4:   for i  $\leftarrow$  0 to n - 8 step 8 do
5:     vKeys  $\leftarrow$  _mm256_loadu(ProbeKeys + i)
6:     vPos  $\leftarrow$  _mm256_and(
7:       _mm256_mullo(vKeys, vPrime), vMask)
8:     vHTKeys  $\leftarrow$  _mm256_i32gather(
9:       HT.Keys, vPos, 4)
10:    matchMask  $\leftarrow$  _movemask(
11:      _mm256_cmpeq(vKeys, vHTKeys))
12:    // Process matches and continue probing
13:  end for
14: end procedure

```

---

#### 8.3.2 Cuckoo Hash Join

Cuckoo hashing [pagh2004cuckoo], as applied to database joins by Polychroniou et al. [22], offers a key advantage for SIMD: each key can only be in one of two positions, enabling parallel lookups without iterative probing. Our imple-

mentation uses two independent hash functions with different primes.

The critical insight is that during the probe phase, we can check *both* possible positions simultaneously using SIMD gather instructions, as shown in Algorithm 4.

---

#### Algorithm 4 AVX2 Cuckoo Hash Join (Probe)

---

```

1: procedure AVX2CUCKOOPEB(ProbeKeys, HT)
2:   for i  $\leftarrow$  0 to n - 8 step 8 do
3:     vKeys  $\leftarrow$  _mm256_loadu(ProbeKeys + i)
4:     // Compute both hash positions in parallel
5:     vPos1  $\leftarrow$  Hash1(vKeys)
6:     vPos2  $\leftarrow$  Hash2(vKeys)
7:     // Gather from BOTH positions simultaneously
8:     vKeys1  $\leftarrow$  _mm256_i32gather(
9:       HT.Keys, vPos1, 4)
10:    vKeys2  $\leftarrow$  _mm256_i32gather(
11:      HT.Keys, vPos2, 4)
12:    // Check matches at both positions
13:    match1  $\leftarrow$  _movemask(
14:      _mm256_cmpeq(vKeys, vKeys1))
15:    match2  $\leftarrow$  _movemask(
16:      _mm256_cmpeq(vKeys, vKeys2))
17:    // Output matched pairs
18:  end for
19: end procedure

```

---

The trade-off is that Cuckoo hashing requires a lower load factor ( $\sim 55\%$ ) and may fail during build if too many displacement chains occur, while Linear Probing handles arbitrary load factors but requires iterative probing.

### 8.4 Experimentation and Results

Experimentation was conducted using randomized parquet integer inputs. We tested our engine on two datasets in particular. The New York Taxi Trip Dataset [21] provides uniform and normally distributed column sets to query. A word frequency count parquet file from Gutenberg project books [23] provides a skewed data set to test our executor. Figure 2 shows the preliminary benchmarks for our SUM and FILTER operators.

As shown in Figure 4, the choice between uniform and skewed data distributions significantly affects query execution across different selectivity configurations. We also have that the queries achieve good average speedup, as seen with aggregations. However the Hash Join implementations didn't seem to yield an important speedup as seen in Figure 6. In Figure 7 we see that all the AVX operations show an improvement in throughput, thus showcasing that the vectorized

versions are more energy efficient in query execution.

## 8.5 Challenges

Initially, our benchmarks showed a negligible reduction in execution time for the AVX2 versions of the algorithms. This was caused by the first revision of the SCAN operator, which dominated the runtime, as it needed to load and decode the compressed blocks from disk. To solve this, we created our own custom benchmark runner which strictly separates the "Compute" phase from the "I/O" phase by pre-loading the Record-Batches [5] into main memory to ensure that performance measurements reflect the real efficiency of the algorithms themselves, rather than disk or memory access latency.

## 9 Conclusion and Future Work

In summary, we have effectively constructed the basis for a vectorized OLAP engine. The results that we obtained from our current implementation confirms that basic aggregations and filter operations can be accelerated by an order of magnitude using explicit SIMD intrinsics. Our results show that SIMD improvements depend on the selectivity of the query on skew data, and

that utilizing SIMD in filter and aggregation operations is a valid choice for building a modern day query engine.

### Future Steps:

1. **Branchless Filter:** Branchless filters using bitmaps rather than a selection vector could yield improvements by reducing memory writes.
2. **Efficient SIMD Hash Join:** Our current implementation of Hash Join doesn't provide any meaningful speedup, a more rigorous implementation could be worth implementing. Or implementing a Nested Loop join to see how SIMD can affect query speed of that operation, although Nested Loop Join is infrequently used in read query engines.
3. **AVX-512 Support:** We intend to explore a newer version of advanced vector extensions called AVX-512 which supports new vector operations that could yield new speedups for query execution.
4. **Intel PCM Monitoring:** Using the Intel PCM [15] monitoring library could give more insights for analysis of the energy efficiency of our algorithms.

## References

- [1] Daniel J Abadi, Samuel R Madden, and Nabil Hachem. "Column-stores vs. row-stores: how different are they really?" In: *Proceedings of the 2008 ACM SIGMOD international conference on Management of data* (2008). Explains the performance mechanics of column stores, relevant to your Arrow/Parquet usage., pp. 967–980.
- [2] ApacheSoftwareFoundation. *Apache Arrow*. (accessed: 11.17.2025). URL: [%5Curl%7Bhttps://arrow.apache.org/%7D](https://arrow.apache.org/).
- [3] ApacheSoftwareFoundation. *Apache Arrow*. Accessed: 2025-11-24. URL: [%5Curl%7Bhttps://arrow.apache.org/%7D](https://arrow.apache.org/).
- [4] ApacheSoftwareFoundation. *Apache Parquet*. URL: [%5Curl%7Bhttps://parquet.apache.org/%7D](https://parquet.apache.org/).
- [5] ApacheSoftwareFoundation. *Arrow RecordBatch API Documentation*. URL: [%5Curl%7Bhttps://arrow.apache.org/docs/cpp/api/table.html#\\_CPPv4N5arrow11RecordBatchE%7D](https://arrow.apache.org/docs/cpp/api/table.html#_CPPv4N5arrow11RecordBatchE%7D).
- [6] Cagri Balkesen et al. "Main-memory hash joins on multi-core CPUs: Tuning to the underlying hardware". In: *2013 IEEE 33rd International Conference on Data Engineering (ICDE)*. Advanced Hash Join optimizations using hardware features, relevant to your AVX join implementation. IEEE. 2013, pp. 362–373.
- [7] Spyros Blanas, Yinan Li, and Jignesh M Patel. "Design and evaluation of main memory hash join algorithms for multi-core CPUs". In: *Proceedings of the 2011 ACM SIGMOD International*

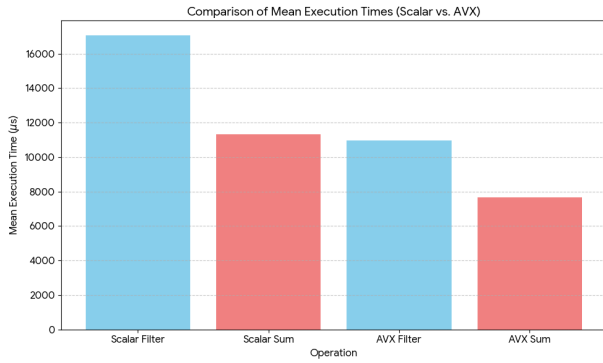


Figure 2: Execution Time (Lower is better)

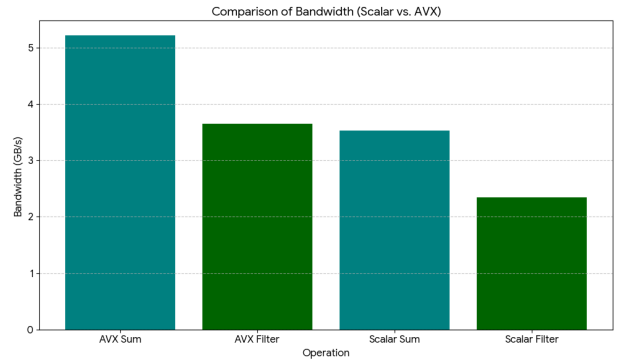


Figure 3: Bandwidth (Higher is better)

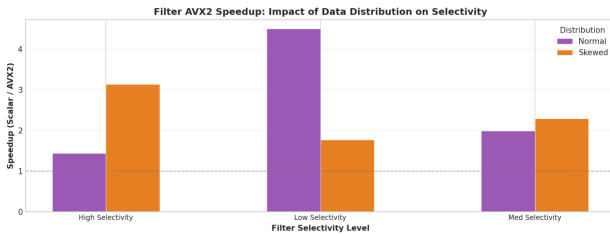


Figure 4: Impact of Data Distribution on Selectivity

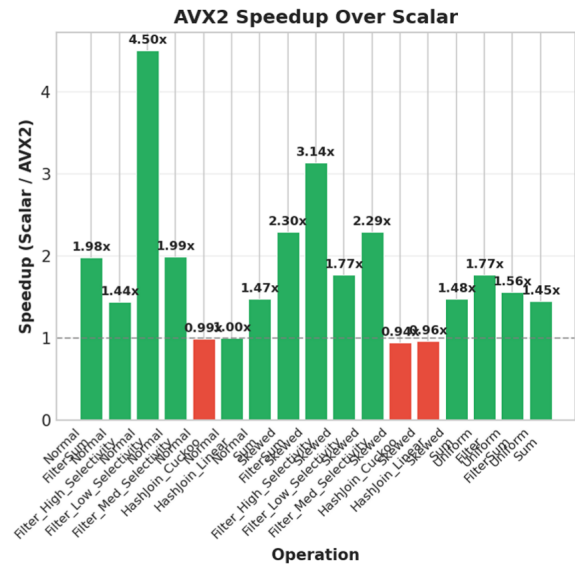


Figure 5: AVX speedup over Scalar

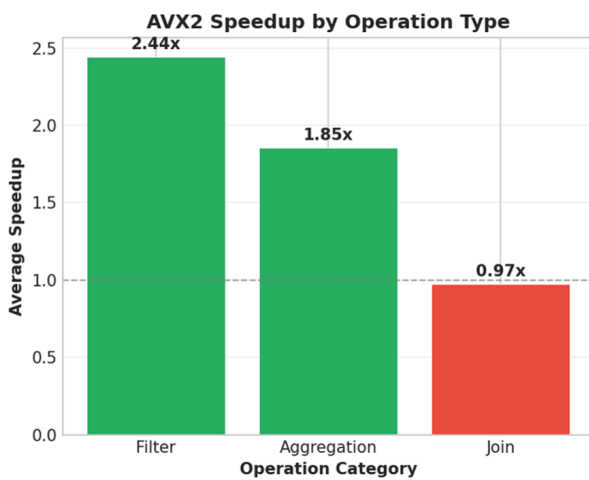


Figure 6: AVX speedup by operation

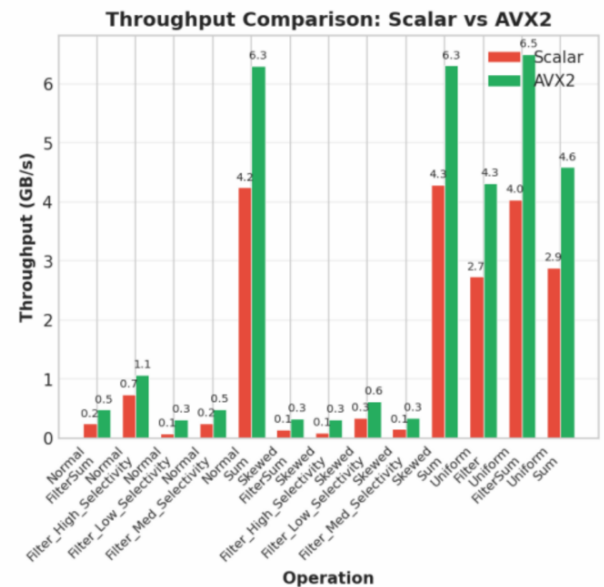


Figure 7: Throughput Comparisons

- Conference on Management of data*. Standard reference for optimizing Hash Joins on modern hardware. 2011, pp. 37–48.
- [8] Peter A. Boncz, Marcin Zukowski, and Niels Nes. “MonetDB/X100: Hyper-Pipelining Query Execution”. In: *CIDR*. Vol. 5. The foundational paper introducing the Vectorized Execution model used in your project. 2005, pp. 225–237.
  - [9] Goetz Graefe. “Volcano - an extensible and parallel query evaluation system”. In: *IEEE Transactions on Knowledge and Data Engineering* 6.1 (2002), pp. 120–135.
  - [10] Goetz Graefe. “Volcano—An Extensible and Parallel Query Evaluation System”. In: *IEEE Transactions on Knowledge and Data Engineering* 6.1 (1994), pp. 120–135.
  - [11] Hiroshi Inoue and Kenjiro Taura. “Faster Set Intersection with SIMD instructions by Reducing Branch Mispredictions”. In: *Proceedings of the 5th Workshop on Systems for Future Multicore Architectures*. 2014.
  - [12] IntelCorporation. *Intel 64 and IA-32 Architectures Optimization Reference Manual*. Latest. Provides intrinsic details and hardware optimization rules. Intel Corporation, 2023.
  - [13] IntelCorporation. *Intel Intrinsics Guide*. (accessed: 11.23.2025). URL: <https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html>.
  - [14] IntelCorporation. *Intel Intrinsics Guide*. Accessed: 2025-11-24. URL: <https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html>.
  - [15] IntelCorporation. *Intel Performance Counter Monitor - A Better Way to Measure CPU Utilization*. (accessed: 11.23.2025). URL: <https://www.intel.com/content/www/us/en/developer/articles/tool/performance-counter-monitor.html>.
  - [16] Changkyu Kim et al. “Sort vs. hash join: Revisiting the trade-offs on modern vector processors”. In: *Proceedings of the VLDB Endowment*. Vol. 2. 1. 2009, pp. 1378–1389.
  - [17] Harald Lang et al. “Make the most out of your SIMD investments: counter control flow divergence in compiled query pipelines”. In: *Proceedings of the 14th International Workshop on Data Management on New Hardware*. 2018, pp. 1–8.
  - [18] Daniel Lemire, Leonid Boytsov, and Nathan Kurz. “SIMD compression and the intersection of sorted integers”. In: *Software: Practice and Experience* 46.6 (2016), pp. 723–749.
  - [19] Yinan Li and Jignesh M Patel. “BitWeaving: fast scans for main memory data processing”. In: *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*. Discusses using bits/masks for scanning, highly relevant to “branchless” filtering. 2013, pp. 289–300.
  - [20] Thomas Neumann. “Efficiently compiling efficient query plans for modern hardware”. In: *Proceedings of the VLDB Endowment*. Vol. 4. 9. Describes the “Compilation” approach (HyPer), useful for contrasting with your “Vectorization” approach. 2011, pp. 539–550.
  - [21] NewYorkCityTaxiAndLimousineCommision. *TLC Trip Record Data*. URL: <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>.
  - [22] Orestis Polychroniou, Arun Raghavan, and Kenneth A Ross. “Rethinking SIMD vectorization for in-memory databases”. In: *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*. 2015, pp. 1493–1508.
  - [23] ProjectGutenberg. *Project Gutenberg*. URL: <https://www.gutenberg.org/>.
  - [24] Mark Raasveldt and Hannes Muehleisen. “DuckDB: an embeddable analytical database”. In: *Proceedings of the 2019 International Conference on Management of Data*. 2019, pp. 1981–1984.
  - [25] Kenneth A Ross. “Selection conditions in main memory”. In: *ACM Transactions on Database Systems (TODS)* 29.1 (2004). Analyzes Branching vs. Branchless logic for Filter operations., pp. 132–161.
  - [26] Juliusz Sompolski, Marcin Zukowski, and Peter Boncz. “Vectorization vs. compilation in query execution”. In: *Proceedings of the 7th International Workshop on Data Management on New Hardware* (2011), pp. 33–40.

- [27] Michael Stonebraker et al. “C-store: a column-oriented DBMS”. In: *Proceedings of the 31st international conference on Very large data bases*. The seminal paper on modern Column-Oriented Storage architectures. 2005, pp. 553–564.
- [28] Dimitris Tsirogiannis, Stavros Harizopoulos, and Mehul A Shah. “Analyzing the energy efficiency of a database server”. In: *Proceedings of the 2010 ACM SIGMOD International Conference on Management of data*. Crucial reference for your ”Energy Efficiency” and power consumption analysis. 2010, pp. 231–242.
- [29] Thomas Willhalm et al. “Vectorizing database column scans with complex predicates”. In: *International Workshop on Data Management on New Hardware*. 2009, pp. 1–7.
- [30] Jingren Zhou and Kenneth A Ross. “Implementing database operations using SIMD instructions”. In: *Proceedings of the 2002 ACM SIGMOD international conference on Management of data*. One of the earliest papers proposing SIMD usage for database operators. 2002, pp. 145–156.